

1) 🌱 **Campus Care – Mental Health Platform for Students** ****Campus Care**** is a comprehensive digital mental health platform tailored for students in higher education. It offers a safe and supportive space for emotional well-being through self-assessments, peer forums, guided exercises, AI assistance, and professional counseling support — all integrated into one accessible system. --- ## 🎥 **Project Demonstration** 📺 ****Watch our full demo on YouTube****: [👉 Click Here to View the Video](https://youtu.be/5fYDuxr0PDE) This video walks through each feature one by one, explaining how it works and the impact it creates for student wellness. --- ## 🌟 **Features** - 🗣️ **Real-time Mood Detection (Voice & Face)** - 📄 **Self-Assessment Tools (PHQ-9, GAD-7, PSS)** - 💬 **Anonymous Peer Support Forum** - 🧘 **Guided Exercises (Yoga, Meditation, Breathing)** - 📖 **Confidential Journaling + AI Insights** - 🎮 **Wellness & Memory Games** - 🤖 **AI Mental Health Chatbot + Emergency SOS** - 📱 **Multilingual Blog Hub & Resource Center** - 📞 **One-Tap Counselor Booking** - 📊 **Admin Analytics Dashboard (De-identified Cohorts)** - 👤 **Role-Based Access: User, Volunteer, Admin** - 🌐 **Fully Responsive & Accessible UI** --- ## 🧠 **Problem Statement** Many students face mental health challenges but lack access to timely, stigma-free support. Traditional systems are fragmented, language-restricted, or overwhelming to navigate—especially in under-resourced institutions. --- ## 🚀 **Our Solution** Campus Care combines mental health assessments, real-time emotional tracking, AI support, wellness content, and community-driven help — all within a private, multilingual platform. We aim to break the stigma, increase access, and promote proactive well-being. --- ## ⚙️ **Tech Stack** #### **Frontend**: - ****Next.js 14 (App Router)**** - ****Tailwind CSS**** + ****ShadCN UI**** - ****Capacitor.js**** (for mobile compatibility) #### **Backend**: - ****Node.js + Express**** - ****PostgreSQL**** (via Neon DB) - ****Prisma ORM**** #### **AI/ML**: - ****Facial Emotion Recognition**** – MediaPipe / Custom ML - ****Voice Mood Detection**** – Wav2Vec + Classifier - ****OpenAI API**** – AI Reports & Chatbot Support --- ## 🛠️ **Installation & Setup** #### **Prerequisites**: - Node.js 18+ - PostgreSQL DB (or Neon) - Capacitor (optional for mobile) #### **Clone & Install**: git clone https://github.com/<your-username>/campus-care.git cd campus-care npm install Setup Environment Create a .env file: DATABASE_URL=your_postgres_connection_string OPENAI_API_KEY=your_openai_key NEXT_PUBLIC_API_URL=http://localhost:5000 Run the App For Frontend (Next.js) npm run dev For Backend (Express) cd backend npm install npm run dev For Mobile (Capacitor) npx cap add android npx cap open android --- ## 📁 **Folder Structure** campus-care/ ├── app/ # Frontend (Next.js) ├── backend/ # Backend (Express.js) ├── prisma/ # Database schema ├── public/ # Static assets ├── components/ # UI Components ├── utils/ # Utility Functions └── README.md --- ## 👥 **Team Members** Anish Agarwal Raj De Modak Amar Pal Aryan Kr Sinha Shashank Shekhar Ananya Mishra --- ## 🔒 **Data Privacy & Ethics** Journaling & mood data are confidential by default Admin dashboard shows only de-identified trends AI tools used only with user consent Designed for privacy-first, student-centric experiences --- ## 📄 **License** This project is currently not licensed. All rights reserved by the authors. Please contact the team for any reuse or contribution

permissions. --- ## 🙌 Acknowledgements This project was built as part of a hackathon to create real-world impact in student mental health through inclusive design, ethical AI, and accessible tech. 🗣️ “Mental health is not a luxury. It's a right. Campus Care brings that right into reach.”

2) 🌐 ConnectHub - Professional Community Platform > A domain-specific professional networking platform that connects verified professionals within their respective fields, fostering meaningful relationships and career opportunities.

[![TypeScript](https://img.shields.io/badge/TypeScript-75%25-blue)](https://www.typescriptlang.org/) [![Java](https://img.shields.io/badge/Java-23.3%25-red)](https://www.java.com/) [![Next.js](https://img.shields.io/badge/Next.js-15.2.4-black)](https://nextjs.org/) [![Spring Boot](https://img.shields.io/badge/Spring%20Boot-3.x-green)](https://spring.io/projects/spring-boot) ## 📄 Table of Contents -

[Overview](#overview) - [Features](#features) - [Tech Stack](#tech-stack) - [Architecture](#architecture) - [Getting Started](#getting-started) - [Configuration](#configuration) - [API Documentation](#api-documentation) - [Security Features](#security-features) - [Contributing](#contributing) ## 🎯 Overview

ConnectHub is a Final Year Project that revolutionizes professional networking by creating domain-specific communities where professionals can connect, share knowledge, collaborate on opportunities, and build authentic relationships within their field. Unlike generic social platforms, ConnectHub ensures meaningful connections by grouping users into profession-based communities. ### Key Highlights - 🎓 **Multi-Domain Support**: Students, Doctors, Engineers, Teachers, Dancers, and 30+ professions - 📧 **Email Verification**: SMS-based OTP authentication via Twilio - 🤖 **AI-Powered Moderation**: OpenAI integration for content safety and fact-checking - 💬 **Real-time Messaging**: WebSocket-based instant messaging with read receipts - 💼 **Job Board**: Profession-specific job postings with safety verification - 🔒 **Trust & Safety**: AI-driven content moderation with auto-delete capabilities ## ✨ Features ### Core Features ##### 1. **Professional Networking** - Create verified professional profiles with bio and profile pictures - Connect with professionals in your specific domain - View global community members by profession - Profile customization with image cropping ##### 2. **Real-time Messaging** - WebSocket-powered instant messaging using STOMP over SockJS - Online/offline status indicators - Message read receipts (single/double check marks) - Image sharing in conversations - Emoji picker integration - Message deletion capabilities - Conversation management #####

3. **Social Feed** - Create and share posts with text and images - Like and comment on posts - AI-powered content moderation with safety notes - Report inappropriate content - Image cropping for post uploads - Community-specific content filtering ##### 4. **Job Board** - Create and browse job postings within your profession - Filter by job type (Full-

time, Part-time, Contract, Internship) - Filter by location (Remote, Hybrid, On-site) - AI-powered link safety verification - Direct application with external links - User-posted job management ##### 5. ****AI Content Moderation**** - Automatic content analysis for: - Sexually explicit content - Hate speech detection - Misinformation flagging - Auto-delete harmful content - Community notes for borderline content - Image content analysis ##### 6. ****Authentication & Security**** - Email/password authentication - SMS OTP verification via Twilio - Session management - Secure password handling ### **User Experience** -  ****Dark/Light Theme****: System-aware theme with manual toggle -  ****Responsive Design****: Mobile-first approach with dedicated mobile navigation -  ****Modern UI****: Built with Radix UI and Tailwind CSS -  ****Smooth Animations****: Framer Motion integration -  ****Toast Notifications****: Real-time feedback with react-hot-toast ##  **Tech Stack** ### **Frontend** - ****Framework****: Next.js 15.2.4 (React 19) - ****Language****: TypeScript - ****Styling****: Tailwind CSS 4.x - ****UI Components****: Radix UI - ****Animations****: Framer Motion - ****State Management****: React Hooks - ****HTTP Client****: Axios - ****WebSocket****: STOMP.js with SockJS - ****Image Handling****: react-image-crop #### **Backend** - ****Framework****: Spring Boot 3.x - ****Language****: Java - ****Database****: JPA/Hibernate (configured) - ****WebSocket****: Spring WebSocket with STOMP - ****AI Integration****: OpenAI API - ****SMS Service****: Twilio API - ****Security****: Spring Security ### **DevOps & Tools** - ****Version Control****: Git/GitHub - ****Package Manager****: npm (Frontend), Maven (Backend) - ****Development****: Turbopack (Next.js) ##  **Architecture** `` Connect_Hub/ |— Frontend/ # Next.js application | |— app/ # Next.js App Router | | |— page.tsx # Landing page | | |— login/ # Authentication | | |— signup/ # Registration | | |— home/ # Main feed | | |— messages/ # Real-time chat | | |— jobs/ # Job board | | |— profile/ # User profiles | |— components/ # Reusable UI components | |— lib/ # API clients and utilities | |— public/ # Static assets | |— backend/ # Spring Boot application | |— profession-connect/ | |— src/main/java/com/community/profession_connect/ | | |— controller/ # REST controllers | | |— service/ # Business logic | | |— AiNoteService.java # AI moderation | | |— model/ # Entity classes | | |— repository/ # Data access layer | |— config/ # Configuration classes | |— .env.example # Environment variables template |— TWILIO_SETUP.md # Twilio configuration guide |— README.md # This file `` ##  **Getting Started** #### **Prerequisites** - ****Node.js**** 20.x or higher - ****Java**** 17 or higher - ****Maven**** 3.8+ - ****PostgreSQL**** or ****MySQL**** (or your preferred database) - ****Twilio Account**** (for SMS OTP) - ****OpenAI API Key**** (for content moderation) #### **Frontend Setup** 1. ****Navigate to the Frontend directory:**** ``bash cd Frontend `` 2. ****Install dependencies:**** ``bash npm install `` 3. ****Create environment file:**** ``bash cp .env.example .env.local `` 4. ****Configure the API endpoint**** in `lib/api.ts`: ``typescript const API\_BASE\_URL = 'http://localhost:8080'; `` 5. ****Run the development server:**** ``bash npm run dev `` 6. ****Open your browser:**** Navigate to http://localhost:3000 #### **Backend Setup** 1. ****Navigate to the backend directory:**** ``bash cd backend/profession-

connect `` 2. ****Configure application properties****
(`src/main/resources/application.properties`): ``properties # Database Configuration
spring.datasource.url=jdbc:postgresql://localhost:5432/connecthub
spring.datasource.username=your_username spring.datasource.password=your_password #
JPA Configuration spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true # Server
Configuration server.port=8080 `` 3. **\*\*Build the project:\*\*** ``bash mvn clean install `` 4.
****Run the application:**** ``bash mvn spring-boot:run `` The backend server will start at
http://localhost:8080 ## 🛠️ Configuration ### Twilio SMS Setup
ConnectHub uses Twilio for SMS-based OTP verification. Follow these steps: 1. ****Sign up for
Twilio**** at https://www.twilio.com 2. ****Copy environment
variables:**** ``bash cp .env.example .env `` 3. ****Configure Twilio credentials**** in `.env`:
``env TWILIO_ENABLED=true TWILIO_ACCOUNT_SID=your_account_sid_here
TWILIO_AUTH_TOKEN=your_auth_token_here TWILIO_PHONE_NUMBER=+15551234567 ``
4. ****For development without SMS****, set: ``env TWILIO\_ENABLED=false `` OTP codes will
be printed to the console. 📄 ****Detailed Setup****: See
[TWILIO_SETUP.md](./TWILIO_SETUP.md) ### OpenAI API Setup For AI-powered content
moderation: 1. ****Get your API key**** from [OpenAI
Platform](https://platform.openai.com/api-keys) 2. ****⚠️ SECURITY WARNING****: Never
commit API keys to version control! 3. ****Configure in backend**** `application.properties`:
``properties openai.api.key=\${OPENAI\_API\_KEY} `` 4. ****Set environment variable:**** ``bash
export OPENAI_API_KEY=your_api_key_here `` ## 📖 API Documentation ###
Authentication Endpoints `` POST /api/auth/signup # User registration POST /api/auth/login
User login POST /api/auth/verify-email # Email verification with OTP `` ### User Endpoints
`` GET /api/users/{id} # Get user profile PUT /api/users/{id} # Update user profile GET
/api/users/search # Search users by profession `` ### Post Endpoints `` GET /api/posts #
Get all posts POST /api/posts # Create new post PUT /api/posts/{id} # Update post DELETE
/api/posts/{id} # Delete post POST /api/posts/{id}/like # Like/unlike post POST
/api/posts/{id}/comment # Add comment `` ### Job Endpoints `` GET /api/jobs # Get all job
posts POST /api/jobs # Create job post DELETE /api/jobs/{id} # Delete job post GET
/api/jobs/filter # Filter jobs by type/location `` ### Messaging Endpoints (WebSocket) `` WS
/ws # WebSocket connection /queue/messages/{userId} # Receive messages /app/send #
Send message /app/mark-read # Mark messages as read `` ## 🛡️ Security Features ###
Content Moderation - AI-powered analysis of all posts - Automatic detection of harmful
content - Categories: Sexually explicit, Hate speech, Misinformation - Auto-delete capability
for severe violations - Community notes for educational purposes ### Job Safety - AI
verification of application links - Safety warnings for suspicious links - User acknowledgment
required for unsafe links ### Authentication - SMS-based OTP verification - Secure session
management - Password encryption ### Best Practices - Environment variables for sensitive
data - API key rotation recommended - Rate limiting (implement as needed) - CORS
configuration for frontend ## 🙌 Contributing This is a Final Year Project. If you'd like to

contribute: 1. Fork the repository 2. Create your feature branch (`git checkout -b feature/AmazingFeature`) 3. Commit your changes (`git commit -m 'Add some AmazingFeature'`) 4. Push to the branch (`git push origin feature/AmazingFeature`) 5. Open a Pull Request ## 📄 License This project is part of an academic Final Year Project. ## 👥 Available Communities ConnectHub supports 30+ professional communities including: - 🎓 Students - 🩺 Doctors - 🏫 Teachers - 🏢 Engineers - ⚖️ Lawyers - 🏗️ Architects - 👨‍🍳 Chefs - 🚔 Police Officers - 🚒 Firefighters - ✈️ Pilots - 🧑‍⚕️ Nurses - 🌾 Farmers - 💃 Dancers - 🎤 Singers - And many more! ## 🌍 Known Issues & Future Improvements - [] Implement rate limiting for API endpoints - [] Add end-to-end testing - [] Enhance mobile responsiveness - [] Add video call functionality - [] Implement notification system - [] Add analytics dashboard ## 📞 Support For issues, questions, or contributions, please open an issue on the GitHub repository. --- ** ⚠️ Important Security Note **: This README previously contained exposed API keys in the repository. All API keys have been identified and should be rotated immediately. Always use environment variables and never commit sensitive credentials to version control. Built with ❤️ as a Final Year Project

3) SocialSphere <div align="center"> <h3> 🚀 A Modern Community Forum & Q&A Platform</h3> <p> Where developers connect, share knowledge, and build communities </p> ![React](https://img.shields.io/badge/React-20232A?style=for-the-badge&logo=react&logoColor=61DAFB) ![TypeScript](https://img.shields.io/badge/TypeScript-007ACC?style=for-the-badge&logo=typescript&logoColor=white) ![Vite](https://img.shields.io/badge/Vite-646CFF?style=for-the-badge&logo=vite&logoColor=white) ![Supabase](https://img.shields.io/badge/Supabase-3ECF8E?style=for-the-badge&logo=supabase&logoColor=white) ![TailwindCSS](https://img.shields.io/badge/Tailwind_CSS-38B2AC?style=for-the-badge&logo=tailwind-css&logoColor=white)
 ![Live Demo](https://img.shields.io/badge/_Live_Demo-FF6B6B?style=for-the-badge)) (https://your-demo-link.com) ![Documentation](https://img.shields.io/badge/_Docs-4ECDC4?style=for-the-badge)) (https://your-docs-link.com) ![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg?style=for-the-badge)) (https://opensource.org/licenses/MIT) </div> --- ## ✨ What Makes SocialSphere Special? SocialSphere isn't just another forum platform—it's a **next-generation community experience** that combines the best of modern web technologies with stunning visual design. Built for developers who demand both functionality and aesthetics. ### 🎯 Why Choose SocialSphere? - 🔒 **Seamless GitHub Integration** - One-click authentication with your GitHub account - 🎨 **Glassmorphism UI** - Beautiful, modern interface with glowing effects - ⚡ **Real-Time Everything** - Live updates, instant notifications, zero refresh needed - 🏗️ **Developer-First** - Built with

TypeScript, optimized for performance -  ****Mobile-Perfect**** - Responsive design that works everywhere --- ##  Tech Stack

 React 18 UI Library	 TypeScript Type Safety	 Vite Build Tool	 Supabase Backend	 Tailwind Styling
--	---	--	---	---

--- ##  Features That Wow ###  ****GitHub-Powered Authentication**** - Seamless OAuth integration - Auto-sync user profiles and avatars - No password hassles, ever ###  ****Rich Content Creation**** - Markdown-supported post editor - Drag-and-drop image uploads - Real-time preview as you type - Code syntax highlighting ###  ****Interactive Voting System**** - Thumbs up/down with visual feedback - Glowing vote indicators - Real-time vote counting - Prevent vote manipulation ###  ****Advanced Commenting**** `` — Original Post | — Comment 1 | — Reply 1.1 | — Reply 1.2 | — Comment 2 | — Reply 2.1 | — Nested Reply `` ###  ****Community Organization**** - Reddit-style communities - Category-based post filtering - Community moderation tools - Custom community themes ###  ****Modern Design Language**** - ****Glassmorphism**** - Frosted glass effects throughout - ****Gradient Borders**** - Subtle glowing animations - ****Dark/Light Modes**** - Seamless theme switching - ****Micro-interactions**** - Delightful hover effects --- ##  Quick Start ### Prerequisites - Node.js 18+ - npm or yarn - Supabase account ### Installation ``bash # Clone the repository git clone https://github.com/ITSAMARHERE/SocialSphere.git # Navigate to project directory cd SocialSphere # Install dependencies npm install # Set up environment variables cp .env.example .env.local # Add your Supabase and GitHub OAuth credentials # Start the development server npm run dev `` ###  Environment Setup ``env VITE_SUPABASE_URL=your_supabase_url VITE_SUPABASE_ANON_KEY=your_supabase_anon_key VITE_GITHUB_CLIENT_ID=your_github_client_id `` --- ##  Screenshots <div align="center"> ###  Homepage ![Homepage]

alt="Image" src="https://github.com/user-attachments/assets/e9d26039-85e0-41e4-bdc9-a625d8718073" /> ### 🗨 Discussion Thread ![Discussion] ### 📱 Mobile Experience

![Image](https://github.com/user-attachments/assets/7247a8da-3dea-49d0-976a-3e1b059feb63) </div> --- ## 🏗 Architecture `` SocialSphere/ | — 🎨 src/ | | — components/ # Reusable UI components | | — pages/ # Route components | | — hooks/ # Custom React hooks | | — services/ # API services | | — types/ # TypeScript definitions | | — utils/ # Helper functions | — 📁 public/ # Static assets | — 🧪 tests/ # Test suites | — 📄 docs/ # Documentation `` --- ## 🤝 Contributing We love contributions! Here's how you can help make SocialSphere even better: ### 🐛 Found a Bug? 1. Check existing issues first 2. Create a detailed bug report 3. Include steps to reproduce ### 💡 Have a Feature Idea? 1. Open a feature request 2. Describe the use case 3. Let's discuss implementation ### 🛠 Want to Code? 1. Fork the repository 2. Create a feature branch 3. Make your changes 4. Add tests if needed 5. Submit a pull request ``bash # Development workflow git checkout -b feature/amazing-feature git commit -m "Add amazing feature" git push origin feature/amazing-feature `` --- ## 📊 Project Stats <div align="center"> </div> --- ## 🗺 Roadmap - [] 🔔 **Push Notifications** - Real-time browser notifications - [] 🎥 **Video Uploads** - Support for video content - [] 🤖 **AI Moderation** - Automated content filtering - [] 📊 **Analytics Dashboard** - Community insights - [] 🌍 **Internationalization** - Multi-language support - [] 📱 **Mobile App** - React Native companion app --- ## 👤 Support & Community <div align="center"> [![Discord](https://img.shields.io/badge/Discord-7289DA?style=for-the-badge&logo=discord&logoColor=white)](https://discord.gg/your-server) [![Twitter](https://img.shields.io/badge/Twitter-1DA1F2?style=for-the-badge&logo=twitter&logoColor=white)](https://twitter.com/your-handle) [![Email](https://img.shields.io/badge/Email-D14836?style=for-the-badge&logo=gmail&logoColor=white)](mailto:your-email@example.com) </div> Got questions? Need help? Found a bug? We're here to help! - 🗨 **Chat with us** on Discord - 📺 **Follow updates** on Twitter - ✉ **Email us** directly - 📖 **Check the wiki** for detailed guides --- ## 📄 License This project is licensed under the MIT License - see the LICENSE file for details. --- <div align="center"> <h3> ⭐ If you like SocialSphere, give it a star!</h3> <p> Built with ❤ by @ITSAMARHERE </p> _{Made possible by amazing open-source projects and the developer community} </div> 4) Mini_Linkedin ✨ A full-stack LinkedIn clone built with the MERN stack, featuring real-time social networking capabilities, professional networking tools, and a modern responsive

design. ## 🚀 Live Demo ****[View Live Application](https://mini-linkedin-frontend-enib.onrender.com/)**** ## 📄 Table of Contents - [Features](#features) - [Tech Stack](#tech-stack) - [Prerequisites](#prerequisites) - [Installation](#installation) - [Environment Variables](#environment-variables) - [Usage](#usage) - [API Endpoints](#api-endpoints) - [Project Structure](#project-structure) - [Screenshots](#screenshots) - [Contributing](#contributing) - [License](#license) ## ✨ Features ### 🔒 Authentication & Security - ****JWT Authentication****: Secure login/logout system with JSON Web Tokens - ****Password Security****: Bcrypt password hashing for enhanced security - ****Protected Routes****: Route protection for authenticated users only - ****Session Management****: Persistent user sessions with token validation ### 👤 User Management - ****Profile Creation****: Complete user profile setup with personal and professional details - ****Profile Updates****: Edit profile information, skills, experience, and education - ****Profile Pictures****: Upload and update profile images with Cloudinary integration - ****User Discovery****: Suggested users feature to help build professional networks ### 🤝 Social Networking - ****Connection System****: Send, accept, and reject connection requests - ****Network Building****: View and manage your professional connections - ****Connection Suggestions****: Algorithm-based user suggestions for networking - ****Connection Status****: Track pending, accepted, and rejected connections ### 📝 Content Management - ****Post Creation****: Create and share professional posts with rich text - ****Image Uploads****: Add images to posts with drag-and-drop functionality - ****Post Interactions****: Like and comment on posts from your network - ****News Feed****: Personalized feed showing posts from connections - ****Post Management****: Edit and delete your own posts ### 📧 Communication - ****Welcome Emails****: Automated welcome emails for new users via Mailtrap - ****Email Notifications****: System notifications for important activities - ****Professional Messaging****: Clean email templates for user communications ### 🎨 User Experience - ****Responsive Design****: Mobile-first design that works on all devices - ****Modern UI****: Clean, professional interface using Tailwind CSS and DaisyUI - ****Smooth Animations****: Subtle animations and transitions for better UX - ****Loading States****: Professional loading indicators and skeleton screens ## 🛠️ Tech Stack ### Frontend - ****React.js****: Frontend framework for building user interfaces - ****Vite****: Fast build tool and development server - ****Tailwind CSS****: Utility-first CSS framework for styling - ****DaisyUI****: Component library built on Tailwind CSS - ****React Router****: Client-side routing for single-page application - ****Axios****: HTTP client for API requests - ****React Query/TanStack Query****: Data fetching and state management ### Backend - ****Node.js****: JavaScript runtime environment - ****Express.js****: Web framework for Node.js - ****MongoDB****: NoSQL database for data storage - ****Mongoose****: MongoDB object modeling for Node.js - ****JWT****: JSON Web Tokens for authentication - ****Bcrypt****: Password hashing library - ****Multer****: Middleware for file uploads - ****Cloudinary****: Cloud-based image and video management ### Additional Services - ****Mailtrap****: Email testing and delivery service - ****Render****: Cloud hosting platform for deployment ## 📋 Prerequisites Before running this application, make sure you have the following installed: - ****Node.js**** (v14.0.0 or later) -

****npm**** (v6.0.0 or later) - ****MongoDB**** (local installation or MongoDB Atlas account) - ****Git**** for version control ## 🌀 Installation ### 1. Clone the Repository ``bash git clone https://github.com/ITSAMARHERE/Mini\_Linkedin.git cd Mini\_Linkedin `` ### 2. Install Dependencies ##### Install Backend Dependencies ``bash npm install `` ##### Install Frontend Dependencies ``bash cd frontend npm install cd .. `` ### 3. Environment Setup Create a `.env` file in the root directory and add the following variables: ``bash # Server Configuration PORT=5000 MONGO_URI=<your_mongo_uri> JWT_SECRET=<your_jwt_secret> NODE_ENV=development MAILTRAP_TOKEN=<your_mailtrap_token> EMAIL_FROM=mailtrap@demomailtrap.com EMAIL_FROM_NAME=<Your_Name> CLOUDINARY_API_KEY=<your_cloudinary_api_key> CLOUDINARY_API_SECRET=<your_cloudinary_api_secret> CLOUDINARY_CLOUD_NAME=<your_cloudinary_cloud_name> CLIENT_URL=http://localhost:5173 `` ### 4. Build and Start the Application ##### Development Mode ``bash # Start backend server npm run dev # In another terminal, start frontend cd frontend npm run dev `` ##### Production Mode ``bash # Build the application npm run build # Start the application npm run start `` ## 🛠 Environment Variables | Variable | Description | Required | Example | |-----|-----|-----|-----| | `PORT` | Server port number | Yes | `5000` | | `MONGO\_URI` | MongoDB connection string | Yes | | | `JWT\_SECRET` | Secret key for JWT tokens | Yes | `your\_jwt\_secret\_key` | | | `NODE\_ENV` | Environment mode | Yes | `development` or `production` | | | `MAILTRAP\_TOKEN` | Mailtrap API token for emails | Yes | `your\_mailtrap\_token` | | | `EMAIL\_FROM` | Sender email address | Yes | `noreply@yourapp.com` | | | `EMAIL\_FROM\_NAME` | Sender name for emails | Yes | `Your App Name` | | | `CLOUDINARY\_API\_KEY` | Cloudinary API key | Yes | `your\_api\_key` | | | `CLOUDINARY\_API\_SECRET` | Cloudinary API secret | Yes | `your\_api\_secret` | | | `CLOUDINARY\_CLOUD\_NAME` | Cloudinary cloud name | Yes | `your\_cloud\_name` | | | `CLIENT\_URL` | Frontend application URL | Yes | `` | ## 🌐 API Endpoints ### Authentication Routes `` POST /api/auth/signup # User registration POST /api/auth/login # User login POST /api/auth/logout # User logout GET /api/auth/me # Get current user `` ### User Routes `` GET /api/users/suggestions # Get suggested users GET /api/users/:id # Get user profile PUT /api/users/profile # Update user profile `` ### Connection Routes `` POST /api/connections/request/:userId # Send connection request PUT /api/connections/accept/:requestId # Accept connection request PUT /api/connections/reject/:requestId # Reject connection request GET /api/connections # Get user connections GET /api/connections/requests # Get connection requests `` ### Post Routes `` GET /api/posts # Get all posts (news feed) POST /api/posts # Create new post GET /api/posts/:id # Get specific post PUT /api/posts/:id # Update post DELETE /api/posts/:id # Delete post POST /api/posts/:id/like # Like/unlike post POST /api/posts/:id/comment # Add comment to post `` ## 📁 Project Structure `` Mini_Linkedin/ ├── backend/ | ├── controllers/ # Route controllers | | ├── auth.controller.js | | ├── connection.controller.js |

| ├── post.controller.js | | └── user.controller.js | | ├── middleware/ # Custom middleware
| | | ├── auth.middleware.js | | | └── multer.middleware.js | | ├── models/ # Database
models | | | ├── User.js | | | ├── Post.js | | | └── Connection.js | | ├── routes/ # API routes |
| | | ├── auth.routes.js | | | ├── connection.routes.js | | | └── post.routes.js | | └──
user.routes.js | | ├── utils/ # Utility functions | | | ├── cloundinary.js | | | ├── db.js | | | └──
sendEmail.js | | └── server.js # Express server setup | ├── frontend/ | | ├── public/ # Static
files | | | ├── src/ | | | | ├── components/ # React components | | | | | ├── common/ # Shared
components | | | | | ├── auth/ # Authentication components | | | | | ├── profile/ # Profile-
related components | | | | | └── posts/ # Post-related components | | | ├── pages/ # Page
components | | | | ├── HomePage.js | | | | ├── LoginPage.js | | | | ├── SignupPage.js | | | |
└── ProfilePage.js | | | └── NetworkPage.js | | | ├── hooks/ # Custom React hooks | | | └──
utils/ # Utility functions | | | ├── context/ # React context providers | | | └── App.js # Main
App component | | ├── index.html | | ├── package.json | | └── vite.config.js | | ├── .env #
Environment variables | | ├── .gitignore | | ├── package.json | | └── README.md `` ## 🧩 Usage
Getting Started 1. **Sign Up**: Create a new account with your email and professional
details 2. **Complete Profile**: Add your profile picture, experience, skills, and education 3.
Connect: Send connection requests to other professionals 4. **Share**: Create posts to
share your professional updates and insights 5. **Engage**: Like and comment on posts
from your network #### Key Features Usage ##### Profile Management - Navigate to your
profile to edit personal information - Upload a professional profile picture - Add your work
experience, education, and skills - Update your headline and summary ##### Building Your
Network - Use the "My Network" section to discover suggested connections - Send
personalized connection requests - Accept or reject incoming connection requests - View
your existing connections ##### Creating Content - Use the post composer on the homepage
- Add text content and images to your posts - Tag connections and add hashtags - Share
professional updates and insights ##### Staying Engaged - Browse your personalized news
feed - Like posts that resonate with you - Leave thoughtful comments on posts - Share
valuable content with your network ## 🚀 Future Enhancements - [] Real-time messaging
system - [] Job posting and application features - [] Company pages and following - []
Advanced search and filtering - [] Video content support - [] Mobile application - []
Analytics dashboard - [] Event creation and management - [] Skill endorsements - []
Content recommendations ## 🐛 Known Issues - Image upload may be slow on slower
connections - Email notifications might end up in spam folders - Mobile responsiveness
needs minor adjustments on some devices ## 🙌 Contributing Contributions are welcome!
Please feel free to submit a Pull Request. For major changes, please open an issue first to
discuss what you would like to change. ### How to Contribute 1. Fork the project 2. Create
your feature branch (`git checkout -b feature/AmazingFeature`) 3. Commit your changes (`git
commit -m 'Add some AmazingFeature'`) 4. Push to the branch (`git push origin
feature/AmazingFeature`) 5. Open a Pull Request ## 🙏 Acknowledgments -

[MongoDB](https://www.mongodb.com/) for the database -
[Cloudinary](https://cloudinary.com/) for image management -
[Mailtrap](https://mailtrap.io/) for email testing - [Tailwind CSS](https://tailwindcss.com/) for styling - [DaisyUI](https://daisyui.com/) for UI components ## 📞 Contact

****Developer****: ITSAMARHERE ****GitHub****:

[@ITSAMARHERE](https://github.com/ITSAMARHERE) ****Project Link****:

https://github.com/ITSAMARHERE/Mini_Linkedin --- ⭐ If you found this project helpful, please give it a star on GitHub! ## 📄

License This project is licensed under the MIT License - see the LICENSE file for details. 5) <h1>Talka</h1> Talka is a real-time chat application built using the MERN stack (MongoDB, Express.js, React.js, and Node.js) along with Socket.io for instant messaging. It's designed to provide a smooth, real-time chatting experience with authentication and scalability in mind. ✨ Features ✅ Real-time messaging – Thanks to Socket.io, messages are instant. ✅ Authentication – Users can sign up and log in securely. ✅ Responsive UI – Works well on both mobile and desktop. ✅ Scalability – Designed to handle a growing number of users. 🚀 Live Demo Experience Talka live:

https://talka.onrender.com/ ⚡ Tech Stack ♦ Frontend: React.js, Redux, CSS Modules ♦ Backend: Node.js, Express.js, MongoDB, Mongoose ♦ WebSockets: Socket.io for real-time communication 🤝 Contributing Want to improve Talka? Contributions are always welcome! Fork the repo Create a new branch Submit a pull request 📄 License This project is under the MIT License. 💡 Feel free to tweak this README.md to better fit the project! 🚀

6) 🛒 E-Commerce Platform A full-stack e-commerce application built with React + Vite on the frontend and Node.js + Express + MongoDB on the backend. The platform provides separate shopping and administration experiences with authentication, product management, cart management, addresses, orders, reviews, search, image uploads, and PayPal checkout. ✨ Features 🛒 Customer shopping experience 🔒 User registration and login 🔑 JWT-based authentication 👤 Role-aware user/admin experience 📦 Product browsing and management 🔍 Product search 🛒 Shopping cart management 📌 Saved customer addresses ⭐ Product reviews and ratings 💳 PayPal payment integration 📄 Order management 🇮🇹 Admin dashboard 🖼️ Product image uploads ☁️ Cloudinary integration 🎨 Responsive UI ⚡ Vite-powered development 🔄 Redux Toolkit state management 🔔 Toast notifications 🎬 UI animations with Framer Motion 🏗️ Architecture The project follows a separate frontend/backend architecture: ECommerce-/ | ├── client/ # React frontend | ├── public/ | ├── src/ | ├── assets/ | ├── components/ | | ├── admin-view/ | | ├── auth/ | | ├── common/ | | ├── shopping-view/ | | | ├── config/ | | ├── lib/ | | ├── pages/ | | ├── admin-view/ | | ├── auth/ | | ├── shopping-view/ | | ├── not-found/ | | ├── unauth-page/ | | | ├── store/ | ├── server/ # Node.js backend |

controllers/ | ├── admin/ | ├── auth/ | ├── common/ | └── shop/ | ├── helpers/ | ├──
 models/ | ├── routes/ | ├── admin/ | ├── auth/ | ├── common/ | └── shop/ | └──
 server.js The repository is explicitly divided into client and server applications. 📦 Tech Stack
 Frontend Technology Purpose React 19 UI development Vite 6 Development server and build
 tooling React Router DOM Client-side routing Redux Toolkit State management React Redux
 Redux integration Axios API communication Tailwind CSS Styling Radix UI Accessible UI
 primitives Framer Motion Animations Lucide React Icons Sonner Toast notifications next-
 themes Theme support The frontend package configuration includes React 19, Redux Toolkit,
 Axios, Tailwind CSS, Radix UI components, Framer Motion, React Router and Vite. Backend
 Technology Purpose Node.js Server runtime Express REST API MongoDB Database
 Mongoose MongoDB ODM JWT Authentication bcryptjs Password hashing Cloudinary
 Image/media storage Multer File uploads PayPal SDK Payment processing CORS Cross-origin
 requests cookie-parser Cookie handling dotenv Environment variables UUID Unique
 identifiers The backend package configuration confirms Express, Mongoose, JWT, bcryptjs,
 Cloudinary, Multer and PayPal dependencies. 🛒 Customer Features Authentication Users
 can: Register an account Login Logout Restore authentication state Access protected pages
 Authenticate API requests using JWT The authentication API exposes: POST
 /api/auth/register POST /api/auth/login POST /api/auth/logout GET /api/auth/check-auth
 These routes are implemented in the authentication router. Passwords are hashed using
 bcryptjs, and successful login generates a JWT containing user identity and role information.
 🛒 Shopping Experience The frontend provides dedicated pages for: /shop/home
 /shop/listing /shop/search /shop/checkout /shop/account /shop/paypal-return
 /shop/payment-success The application's main router separates authentication, admin and
 shopping experiences through dedicated layouts and route guards. Product Discovery
 Customers can: Browse products View product listings Search products Filter/discover
 products View product information Add products to cart 🛒 Shopping Cart The cart system
 supports: Adding products Updating quantities Removing products Viewing cart contents
 Proceeding to checkout The backend contains a dedicated cart controller and route:
 /api/shop/cart The shop API is separated into individual domains for products, cart,
 addresses, orders, search and reviews. 📍 Address Management Customers can manage
 saved shipping addresses through: /api/shop/address This allows the checkout flow to
 associate an order with customer address information. ⭐ Reviews & Ratings The
 application contains: Product reviews Rating functionality Star-rating UI Review API
 endpoints Backend review functionality is handled through: /api/shop/review with a
 dedicated review controller and Review MongoDB model. 💳 PayPal Checkout The
 application integrates PayPal into the order workflow. The checkout flow works
 approximately like this: Customer Cart | ▼ Checkout | ▼ Create Order | ├── Create PayPal
 order | ├── Save order in MongoDB | └── Return approval URL | ▼ PayPal Checkout | ▼
 Capture Payment | └── ┌───┴───┐ ▼ ▼ Confirm Order Update Stock The backend
 creates a PayPal order using the cart items, stores the resulting order, and returns a PayPal

approval URL to the frontend. After payment capture, the implementation updates the order/payment status and decreases product stock. 👤 **Admin Dashboard** The project includes a dedicated admin experience. Admin routes include: /admin/dashboard /admin/features /admin/products /admin/orders The frontend also contains dedicated admin components for: Sidebar/navigation Product management Product tiles Image uploading Orders Order details Admin layout The main application router defines these admin routes separately from the customer shopping routes. 📦 **Product Management** Administrators can manage the product catalog through the admin interface. The backend separates product operations into: /api/admin/products /api/shop/products This keeps administrative product operations separate from customer-facing product APIs. 📋 **Order Management** Orders are handled through dedicated APIs for both customers and administrators. Customer order functionality: /api/shop/order Admin order functionality: /api/admin/orders The backend contains dedicated controllers and models for order processing. 🗄️ **Database Models** The application currently contains the following Mongoose models: User Product Cart Address Order Review Feature These models represent the main entities of the e-commerce platform. User Stores customer/admin account information and authentication-related data. Product Stores product catalog information such as product details, pricing and inventory. Cart Stores customer cart information and selected products. Address Stores customer shipping/address information. Order Stores order items, customer/address information, payment information and order status. Review Stores customer product reviews and ratings. Feature Stores configurable store feature information. 🚀 **API Structure** The Express server registers the following API groups: /api/auth /api/admin/products /api/admin/orders /api/shop/products /api/shop/cart /api/shop/address /api/shop/order /api/shop/search /api/shop/review /api/common/feature The server configures CORS, JSON parsing, cookies and MongoDB before registering these route modules. 🔒 **Authentication Flow**

```

graph LR
    User --> Register
    Register --> bcrypt[bcrypt Password]
    bcrypt --> Hashing
    Hashing --> MongoDB[MongoDB User Login]
    MongoDB --> Verify[Verify Password]
    Verify --> JWT[Generate JWT]
    JWT --> Frontend
    Frontend --> Auth[Auth Middleware]
    Auth --> Protected[Protected API]
  
```

Authorization: Bearer <token> ▼ Protected API The backend authentication middleware reads the bearer token from the Authorization header and verifies it with JWT. ⚙️ **Getting Started**

Prerequisites Install/configure: Node.js npm MongoDB PayPal developer credentials Cloudinary account

1. Clone the Repository `git clone https://github.com/ITSAMARHERE/ECommerce-.git` `cd ECommerce-`
2. Install Frontend Dependencies `cd client` `npm install`
3. Install Backend Dependencies Open another terminal: `cd server` `npm install`

🔑 **Environment Variables** Create: `server/.env` Example: `PORT=5000` `MONGO_URL=your_mongodb_connection_string` `CLIENT_BASE_URL=http://localhost:5173` `# PayPal PAYPAL_CLIENT_ID=your_paypal_client_id`

PAYPAL_CLIENT_SECRET=your_paypal_client_secret # Cloudinary
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_API_SECRET=your_cloudinary_api_secret Use the exact environment variable names expected by the corresponding helper/controller files in the repository. Never commit .env files or credentials to GitHub. ▶ Running the Project Start Backend cd server npm run dev The backend uses: http://localhost:5000 by default unless another PORT is configured. Start Frontend Open another terminal: cd client npm run dev Vite normally serves the application at: http://localhost:5173 🌈 Available Scripts Client npm run dev Start the Vite development server. npm run build Create a production build. npm run preview Preview the production build. npm run lint Run ESLint. These scripts are defined in the frontend package configuration. Server npm run dev Start the server using Nodemon. npm start Start the backend server. The current backend package also contains a placeholder test script rather than an automated test suite. 📁 Important Directories client/src/components Reusable UI components. admin-view/ auth/ common/ shopping-view/ client/src/pages Application-level pages. admin-view/ auth/ shopping-view/ not-found/ unauth-page/ client/src/store Redux Toolkit state management. server/controllers Business logic separated by domain: admin/ auth/ common/ shop/ server/routes Express API routes separated by domain: admin/ auth/ common/ shop/ server/models MongoDB/Mongoose schemas. User.js product.js Cart.js Address.js Order.js Review.js Feature.js server/helpers Reusable backend integrations and helper functionality, including payment-related functionality. 🔒 Security Considerations This project demonstrates a complete full-stack e-commerce workflow, but before using it in a production environment, additional security hardening is recommended. Recommended improvements Move JWT signing secrets into environment variables. Use a strong production-only JWT secret. Add request validation. Add API rate limiting. Add stronger admin authorization middleware. Validate uploaded file types and sizes. Add centralized API error handling. Use HTTPS in production. Configure production CORS origins explicitly. Add automated security testing. Add payment verification and failure/rollback handling. Protect sensitive endpoints against abuse. 🚧 Future Improvements Potential improvements include: Automated unit tests Integration/API tests Request schema validation Centralized error handling Pagination for products/orders Advanced product filtering Wishlist functionality Coupon/discount system Product categories Order cancellation Refund management Email order notifications Inventory reservation Improved payment failure handling CI/CD pipeline Docker support Production deployment documentation 🤝 Contributing Contributions are welcome. 1. Fork the repository 2. Create a feature branch git checkout -b feature/your-feature 3. Make your changes 4. Run checks cd client npm run lint npm run build 5. Commit your changes git commit -m "feat: add your feature" 6. Push your branch git push origin feature/your-feature 7. Open a Pull Request 📄 License The backend package.json currently declares the project under the ISC license. For clearer open-source distribution, consider adding a root-level

LICENSE file. 🧑‍💻 Author Amar Pal GitHub:

[@ITSAMARHERE](https://github.com/ITSAMARHERE) 🔗 Repository GitHub:

[https://github.com/ITSAMARHERE/ECommerce-

](https://github.com/ITSAMARHERE/ECommerce-) ⭐ If you find this project useful

Consider giving the repository a ⭐ on GitHub and feel free to contribute improvements.